ChatGPT

LangChain Mastery: Phase 2 – Core Concepts

Runnables & LangChain Expression Language (LCEL)

Concept and Theory: LangChain introduces the *LangChain Expression Language (LCEL)* to simplify building complex AI pipelines. LCEL uses a **pipe-style syntax** (using `|` or `.pipe()` / `.invoke()`) to link components (prompts, models, parsers, etc.) in sequence, making code concise and easy to read【1†L27-L36】【31†L473-L481】. Each component in an LCEL pipeline is called a **Runnable**. For example, you can pipe a prompt to a model to an output parser, and the output of each step feeds into the next. This is analogous to an assembly line: each station (Runnable) takes an input, processes it, and hands off its output to the next station.

Common **Runnable** types include:

- **RunnableSequence**: Chains multiple steps in order. Its output flows from one step to the next【31†L473-L481】【47†L162-L170】.
- **RunnableParallel**: Runs multiple operations **in parallel** on the same input or different inputs, returning all results at once【52†L18-L23】. Think of it as duplicating the input to two workers and running them concurrently.
- **RunnableBranch**: Chooses a branch based on a condition. It checks each condition and runs the corresponding runnable, similar to a `switch` or `if-else` statement【18†L55-L63】.
- **RunnableLambda**: Wraps a custom function (e.g. JavaScript function) as a runnable step.
- **RunnablePassthrough**: Forwards the input unchanged to the next step (useful when you need to pass along a value while performing other parallel tasks)【52†L18-L23】.

LCEL is "minimalist" and supports advanced features like streaming, async, and parallel execution【1†L27-L36】【52†L18-L23】. It complements LangChain's chains by making them easier to write. For example, instead of manually calling each component, you can write `prompt.pipe(llm).pipe(parser)` to create a single pipeline【31†L473-L481】【47†L162-L170】. This abstraction speeds up development and keeps the code organized.

Practical Lab: Let's build and test some runnables step by step. Assume you've installed LangChain (`npm install langchain @langchain/openai`) and set `OPENAI_API_KEY` in your `.env`.

1. **Simple RunnableSequence:** Create a JavaScript file (`runnable_lab.js`) and import necessary classes:

```
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { RunnableSequence } from "@langchain/core/runnables";
import { ChatOpenAI } from "@langchain/openai";
import { StringOutputParser } from "@langchain/core/output_parsers";
```

```
import * as dotenv from "dotenv";
dotenv.config();
```

- We import a prompt template, sequence runnable, an OpenAI chat model, and a parser for output.
- `dotenv.config()` loads the API key from `.env`.

2. **Define a Prompt:** Create a prompt template that asks the LLM to explain a topic:

```
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a helpful assistant."],
  ["human", "{inputText}"]
]);
```

- This defines a conversation with a system message and a placeholder `{inputText}` for user input.

3. **Initialize the Model:** Use ChatOpenAI (e.g. GPT-4) with low temperature for deterministic answers:

```
const model = new ChatOpenAI({ modelName: "gpt-4", temperature: 0 });
```

- `temperature: 0` makes the output more focused and repeatable.

4. **Set Up a String Parser:** Prepare a parser to extract plain text:

```
const parser = new StringOutputParser();
```

- This just takes the raw model response and returns its text.

5. **Compose a RunnableSequence:** Now chain the prompt, model, and parser in sequence:

```
const chain = RunnableSequence.from([
  prompt,
  model,
  parser
]);
```

- This creates a pipeline: take input -> apply `prompt.format` -> pass formatted prompt to `model.invoke()` -> pass model output to parser.
- Under the hood, `RunnableSequence.from([...])` wires these steps so that each feeds into the next【47†L162-L170】.

6. **Run the Chain:** Invoke the chain with a specific input:

```
const runChain = async () => {
  const result = await chain.invoke({ inputText: "Explain recursion in
simple terms." });
  console.log("LLM Answer:", result);
};
runChain();
```

   - `chain.invoke({ inputText: ... })` supplies the variable to the prompt.
   - It returns an object (or simple string) with the parser output. We log it.
   - Expect the console to print a concise explanation of recursion.

7. **Verify:** If everything is correct, you should see the LLM's answer in your console. If you get an error like "Expected a Runnable... got something else," check that each element in the sequence is a proper Runnable or PromptTemplate【12†L0-L4】. Common mistakes include forgetting to `.pipe()` a prompt or passing raw strings instead of variables.

8. **RunnableParallel Example:** Now try running model predictions in parallel. For instance, ask the same question with two different temperatures:

```
import { RunnableParallel } from "@langchain/core/runnables";

const modelHigh = new ChatOpenAI({ modelName: "gpt-4", temperature: 0.9 });
const modelLow  = new ChatOpenAI({ modelName: "gpt-4", temperature: 0 });

const parallel = RunnableParallel.from({
  answerLow: modelLow,
  answerHigh: modelHigh
});

const answer = await parallel.invoke("Tell me a joke about robots.");
console.log(answer);
```

   - Here `RunnableParallel.from({...})` runs `modelLow.invoke(input)` and `modelHigh.invoke(input)` concurrently【52†L18-L23】.
   - The result will be an object `{ answerLow: <low-temp answer>, answerHigh: <high-temp answer> }`.
   - This demonstrates using multiple LLM calls in parallel and comparing outputs.

9. **RunnableBranch Example:** Create two simple chains and route based on a keyword. For example, respond differently if input contains "weather":

```
import { RunnableBranch, RunnableSequence } from "@langchain/core/
runnables";
import { RunnablePassthrough } from "@langchain/core/runnables";

const weatherPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a weather bot. Answer questions about weather."],
  ["human", "{question}"]
]);

const generalPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a general helper."],
  ["human", "{question}"]
]);

const weatherChain = RunnableSequence.from([weatherPrompt, model, parser]);
const generalChain = RunnableSequence.from([generalPrompt, model, parser]);

const branch = RunnableBranch.from([
  [
    (data) => data.topic && data.topic.toLowerCase().includes("weather"),
    weatherChain
  ],
  generalChain  // default
]);

const fullChain = RunnableSequence.from([
  { topic: weatherChain, question: new RunnablePassthrough() },
  branch
]);

// Test it:
const out1 = await fullChain.invoke({ topic: "Today's weather", question:
"Will it rain tomorrow?" });
const out2 = await fullChain.invoke({ topic: "Cooking tips", question:
"How to make pasta?" });
console.log("Weather branch answer:", out1);
console.log("General branch answer:", out2);
```

- We build two chains: `weatherChain` and `generalChain`. Then define a `RunnableBranch` that checks a condition (`data.topic.includes("weather")`) 【18†L55-L63】. If true, it runs `weatherChain`; otherwise `generalChain` (default).
- The `RunnableSequence` labeled with `{topic: weatherChain, question: RunnablePassthrough}` is a way to feed the topic into classification step, but here simplified: we directly give both fields.
- Testing shows that a weather question triggers the weather bot, else the general bot.

Common Pitfalls: Forgetting to `.pipe()` components or to use `await` on `.invoke()` will cause runtime errors. Ensure that all pieces (prompts, models, parsers) are valid Runnables. When using `RunnableParallel` or `RunnableBranch`, double-check how your data is packaged (often as objects with named keys).

Knowledge Check:

- *Key Takeaways:* LCEL lets you **chain components** with pipes. A `RunnableSequence` connects steps end-to-end【47†L162-L170】. `RunnableParallel` runs branches in parallel【52†L18-L23】. `RunnableBranch` routes based on conditions【18†L55-L63】. These abstractions make code readable and modular.
- *Summary:* Runnables are **building blocks** in LangChain. LCEL syntax makes pipelines concise. Think of a Runnable as a function you can chain (`|`) or sequence (`.pipe()`).
- *Practice Question:* What is the difference between a RunnableSequence and a RunnableParallel? *(Answer: Sequence runs steps in order, Parallel runs multiple steps at once on the same input.)*
- *Exercise:* Modify the above code to add a third chain (e.g., a "math" chain) using `RunnableBranch`, so that if the topic includes "calculate", it uses a different LLM prompt to solve math questions.
- *Mini Project:* Create a chain that takes a user's question, concurrently generates both a short answer and a list of relevant keywords (using two parallel branches), then formats both results into a JSON object via an output parser. Verify that both pieces come from the same original question.

Chains

Concept and Theory: In LangChain, a **Chain** is a series of steps (prompts, models, transformations) that together process an input to produce an output. Each step's output feeds into the next step as input 【31†L473-L481】【47†L162-L170】. In practice, Chains let you break complex tasks into simpler subtasks. For example, you might first translate text, then summarize it. Under the hood, Chains often use Runnables (e.g. `RunnableSequence`) to implement this pipelining. Chains make logic explicit and debuggable: you see each step and its role.

A simple chain might be "Prompt → LLM → Parser"【31†L473-L481】. More advanced chains use parallel or conditional steps internally. LangChain also provides high-level Chain classes (like `LLMChain` or retrieval chains), but in LCEL style, you can often just compose your own with `RunnableSequence.from([...])`.

Practical Lab: Let's build and experiment with a few chain patterns. Continuing from above (`model` and `parser` already defined):

1. **Simple Sequential Chain:** We did this in the Runnables lab. As a quick example, use a `RunnableSequence` with a prompt and model to translate English to French:

```
const translatePrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a helpful translator."],
  ["human", "Translate the following to French: {englishText}"]
]);
const translateChain = RunnableSequence.from([translatePrompt, model,
```

```
parser]);
const translation = await translateChain.invoke({ englishText: "Good
morning" });
console.log("Translation:", translation);
```

- Expects a French response. This shows a one-input, one-output chain.

2. **Parallel Chains (Multiple Outputs):** Suppose you have one document and want both its **title** and **summary**. You can use two chains in parallel:

```
const titlePrompt = ChatPromptTemplate.fromMessages([
  ["system", "Summarize the text in one sentence as a title."],
  ["human", "{doc}"]
]);
const summaryPrompt = ChatPromptTemplate.fromMessages([
  ["system", "Summarize the text in a short paragraph."],
  ["human", "{doc}"]
]);

const titleChain = RunnableSequence.from([titlePrompt, model, parser]);
const summaryChain = RunnableSequence.from([summaryPrompt, model, parser]);

const parallelChains = RunnableParallel.from({
  title: titleChain,
  summary: summaryChain
});

const doc = "LangChain is an AI framework that simplifies integrating LLMs
into applications...";
const results = await parallelChains.invoke(doc);
console.log("Title:", results.title);
console.log("Summary:", results.summary);
```

- We run two independent chains on the same `doc` input. The output is an object with both results.

3. **Conditional Chain (Branching):** We already saw `RunnableBranch`. Here's another structure: first classify a question, then pick a specialized chain. For instance, if a question mentions "math", use a math helper; else use a general answer:

```
// Classification prompt (from previous example)
const classPrompt = ChatPromptTemplate.fromMessages([
  ["system", "Classify the question topic."],
  ["human", "Question: {question}\nCategory:"]
]);
```

```
const classificationChain = RunnableSequence.from([classPrompt, model,
parser]);

// Create specialized chains
const mathPrompt = ChatPromptTemplate.fromMessages([
  ["system", "Solve the math problem:"],
  ["human", "{question}"]
]);
const mathChain = RunnableSequence.from([mathPrompt, model, parser]);

const generalPrompt = ChatPromptTemplate.fromMessages([
  ["system", "Answer the question to the best of your ability:"],
  ["human", "{question}"]
]);
const generalChain = RunnableSequence.from([generalPrompt, model, parser]);

// Branch logic: if classification says "MATH", use mathChain
const questionBranch = RunnableBranch.from([
  [
    data => data.classification && data.classification === "MATH",
    mathChain
  ],
  generalChain  // default
]);

// Full chain: first classify, then branch
const fullChain = RunnableSequence.from([
  classificationChain,
  questionBranch
]);

const outMath = await fullChain.invoke({ question: "What is 5+7?" });
const outGen  = await fullChain.invoke({ question: "Who was Einstein?" });
console.log("Math answer:", outMath);
console.log("General answer:", outGen);
```

    - Here, we used a small classification step (just as an example) and routed accordingly.

4. **Nested Chains:** You can also *nest* chains. For example, inside one chain, you might run another chain via a Runnable or function. This is advanced, but conceptually it means you can call `await someChain.invoke()` inside another chain or a `RunnableLambda`. (Advanced users can explore `RunnableLambda` to wrap custom JS code within a chain pipeline.)

Code & Commands Explanation: In these examples, note the pattern: we import what we need, define prompt templates with placeholders, create chains with `RunnableSequence.from([...])`, and call them with `.invoke({ inputs })`. Each `invoke()` runs the entire pipeline.

- `RunnableSequence.from([...])` takes an **array of runnables** (prompts, models, parsers, etc.) and connects them.
- The final output (often from a parser) is returned by `.invoke()`.
- We usually `await` `.invoke()` since it's asynchronous (it calls an API under the hood).

Knowledge Check:

- *Key Takeaways:* A **Chain** is a pipeline of steps (prompt, LLM, parser)【31†L473-L481】【47†L162-L170】. You can run chains in **parallel** to get multiple outputs from one input. You can also chain **sequential** operations or **conditional** branches to handle different cases.
- *Summary:* Chains structure the workflow: they break tasks into modular steps. Each step can be tested and understood in isolation, making debugging easier. LangChain gives you the building blocks (runnables) to compose complex chains.
- *Practice Question:* Given the `RunnableParallel` example above, what will be in the `results` object if you invoked it? *(Answer: an object with keys* `title` *and* `summary`*, each containing the respective text.)*
- *Exercise:* Modify the title/summary example to also produce a list of **keywords** from the document (hint: create a third branch in the parallel).
- *Mini Project:* Build a nested chain: for example, first detect the language of an input (chain 1), then if it's not English, translate it to English (chain 2), and finally summarize it (chain 3). Use chains inside chains or a combination of `RunnableSequence` and `RunnableBranch`.

Memory

Concept and Theory: By default, LLM API calls are **stateless** – each call knows nothing of previous conversation. *Memory* components let your app remember context across turns, making chatbots conversational. LangChain offers memory types like **ConversationBufferMemory**, **WindowMemory**, and **SummaryMemory**【44†L61-L67】【22†L34-L42】:

- **ConversationBufferMemory:** Stores the *entire* chat history. It is the simplest memory — it just records everything said so far【44†L61-L67】.
- **ConversationBufferWindowMemory:** Keeps only the last *k* messages (a sliding window). This is useful to limit context length; older messages beyond `k` are dropped【44†L61-L67】.
- **ConversationSummaryMemory:** Maintains a concise summary of the conversation instead of all details. The system (typically another LLM call) generates and updates a summary over time【44†L61-L67】.

Memory lets the LLM refer back to earlier points. For example, if the user says "I have a dog named Max" and later asks "What's his favorite game?", a memory-enabled bot can recall "Max" is the dog【22†L34-L42】. Without memory, it would have no context.

Practical Lab: Let's simulate memory in a chatbot. (LangChain's built-in memory classes have evolved, so we'll use a simple manual approach with our chain to illustrate the concept.)

1. **Setup:** Use the same `ChatOpenAI` model and a chat-like prompt template:

```
const chatPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a friendly AI assistant."],
  ["human", "{input}"]
]);
const chatChain = RunnableSequence.from([chatPrompt, model, parser]);
```

   - This chain expects an `input` variable and returns the LLM's reply.

2. **Simulate Conversation Memory:** Create a function to track history:

```
let history = "";  // memory buffer as a string

async function askUser(question) {
  // Include conversation history in the prompt
  const fullInput = history + `User: ${question}\nAI:`;
  const response = await model.invoke(fullInput);
  const answer = response.content.trim();
  // Update history with both user question and AI answer
  history += `User: ${question}\nAI: ${answer}\n`;
  return answer;
}
```

   - Here we **prepend** the entire `history` to each prompt, then append the new exchange. This mimics `ConversationBufferMemory` (full history)【44†L61-L67】.
   - If history gets very long, later turns include more text, simulating how context grows.

3. **Test Full Memory:** Ask a sequence of related questions:

```
console.log(await askUser("Hi, I have a dog named Bella."));
console.log(await askUser("What is her breed?"));
```

   - The first answer should greet or say something about Bella. The second should (ideally) use the fact that Bella is a dog to respond.
   - If the model knows Bella is a dog from context, it will answer accordingly. If not, we might not see correct behavior (debug by printing `history`).

4. **Window Memory Example:** Modify to keep only the last 2 interactions:

```
const maxTurns = 2;
async function askUserWindow(question) {
  // Only use the last maxTurns lines from history
  const parts = history.split("\n");
  const recent = parts.slice(- (2 * maxTurns)).join("\n");
  const fullInput = recent + `\nUser: ${question}\nAI:`;
  const response = await model.invoke(fullInput);
  const answer = response.content.trim();
  history += `User: ${question}\nAI: ${answer}\n`;
  return answer;
}
```

- Here, we reconstruct `recent` history by slicing the last few lines, dropping older lines 【44†L61-L67】.
- Now ask again:

```
console.log(await askUserWindow("Hello again, AI."));
console.log(await askUserWindow("Do you remember my dog's name?"));
```

- With window memory of size 2 turns, the second question should still see the mention of the dog if it falls within the last 2 exchanges. If it falls out, the model might not recall Bella by name.

5. **Summary Memory Example:** As conversation grows, summarizing can save tokens. We can create a summary manually:

```
async function summarizeHistory() {
  const summaryPrompt = ChatPromptTemplate.fromMessages([
    ["system", "Summarize the following conversation."],
    ["human", "{historyText}"]
  ]);
  const summaryChain = RunnableSequence.from([summaryPrompt, model,
parser]);
  const summary = await summaryChain.invoke({ historyText: history });
  console.log("Conversation summary:", summary);
  // Replace history with summary to reduce length
  history = `Summary of previous chat: ${summary}\n`;
}

// Example: after a few Q&A
console.log(await askUser("I like basketball."));
console.log(await askUser("I also love pizza."));
await summarizeHistory();
```

```
console.log("After summarizing, history is reset to summary.");
console.log(await askUser("What sports do I like?"));
```

- This calls the model to summarize the chat so far, then replaces `history` with that summary. Now future prompts include the summary as context.
- This mimics `ConversationSummaryMemory` 【44†L61-L67】: only an outline of the conversation is kept.

Expected Results:

- With *buffer memory*, the AI should correctly refer back to earlier details (e.g., recalling Bella the dog).
- With *window memory*, if the relevant fact falls outside the window, the AI may "forget" it.
- With *summary memory*, the AI only has the condensed version, so it may answer based on the summary instead of raw chat.

Common Mistakes: Forgetting to include the memory in prompts will always yield stateless responses. Also, make sure the memory format (string, JSON, etc.) matches what the prompt expects (we used plain text chat format). If the LLM outputs unexpected content, check the prompt structure and variables.

Knowledge Check:

- *Key Takeaways:* Memory components let your chatbot **remember context** across turns【22†L34-L42】【44†L61-L67】. Buffer memory saves everything, window memory saves only the last *N* exchanges, and summary memory saves a compressed version. This enables multi-turn conversation awareness.
- *Summary:* Without memory, each LLM call is isolated. By building a memory buffer (string, database, or special LangChain memory), we feed past chat into the prompt so the LLM "remembers"【22†L34-L42】.
- *Practice Question:* In our buffer memory example, what would happen if we ask "Do you remember my favorite food?" without including the initial mention? *(Answer: The model will have no memory of earlier mention and likely ask for clarification.)*
- *Exercise:* Implement a simple "custom memory" that stores the user's name. For example, if the user says "My name is X," your memory should record this and insert "User: X" in future prompts to be addressed by name.
- *Mini Project:* Build a chatbot that greets the user by name and remembers their preferences (e.g., "Your favorite color is blue"). Use either manual history or integrate with a database (Redis) for persistent memory. After each answer, check that the chatbot correctly recalls the stored information.

Final Project – Core Phase Integration

To consolidate these concepts, build an **end-to-end chatbot application** using Node.js/TypeScript and LangChain that combines runnables, chains, and memory:

1. **Scenario:** Create a multi-turn personal assistant bot. It should:

2. **Greet** the user and ask their name. Store this in memory.

3. Be able to **answer general questions** (using an LLM chain).
4. If asked about the weather or calculations, route to specialized sub-bots using **RunnableBranch** logic.
5. **Remember** personal preferences: if the user later mentions something (hobby, pet, etc.), the bot should recall it in future responses.

6. Support **streaming** answers (bonus: print the answer token-by-token in console as it's generated).

7. **Implementation Steps:**

8. **Set up the base chain:** A `RunnableSequence` with a chat prompt and model.

9. **Memory integration:** At each user turn, append to a history string (or use a Map for named values like name, pet). Inject this history into the prompt for context【22†L34-L42】.
10. **Name storage:** Detect phrases like "My name is ___" using simple parsing or a prompt, then store that name in memory.
11. **Branching:** If the user asks "What's the weather?", use a weather tool or a web API via a custom `RunnableLambda` tool. If the user asks a math question, use a calculator tool (LangChain provides a calculator tool class)【18†L185-L193】. Otherwise use the general chat LLM.
12. **Demo conversation:** Show a conversation sequence, e.g.:
    - User: "Hi" → Bot: "Hello! What's your name?"
    - User: "I'm Alice." → Bot: "Nice to meet you, Alice. How can I assist you?"
    - User: "What's 2+2?" → Bot calculates via tool and answers.
    - User: "Thanks!" → Bot responds.
    - User: "What's my name?" → Bot says "Your name is Alice," using memory.
    - User: "Tell me a joke." → Bot tells a joke (general chain).

13. **Ensure smooth flow:** Check logs or use LangSmith (if available) to trace each step.

14. **What to Submit:** Document the code, and include a sample conversation transcript. Explain how each part works:

15. How runnables/chains are set up.

16. How memory is recorded and used.
17. How conditional branching was implemented.

This comprehensive project demonstrates Runnables and LCEL syntax in action, plus chains and memory working together to create an intelligent, context-aware assistant.

Sources: Concepts for chaining and LCEL drawn from LangChain documentation【31†L473-L481】【47†L162-L170】. Memory discussion adapted from LangChain's conversational memory tutorial【22†L34-L42】【44†L61-L67】 and documentation. The branching example is inspired by LangChain examples【18†L55-L63】【52†L18-L23】. All code samples are illustrative of LangChain.js usage.
